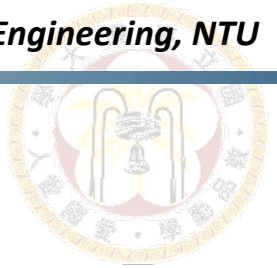
Advanced Computer-Aided VLSI System Design

Midterm Project:
ViT Patch Embedding Accelerator

Graduate Institute of Electronics Engineering, National Taiwan University



NTU GIEE

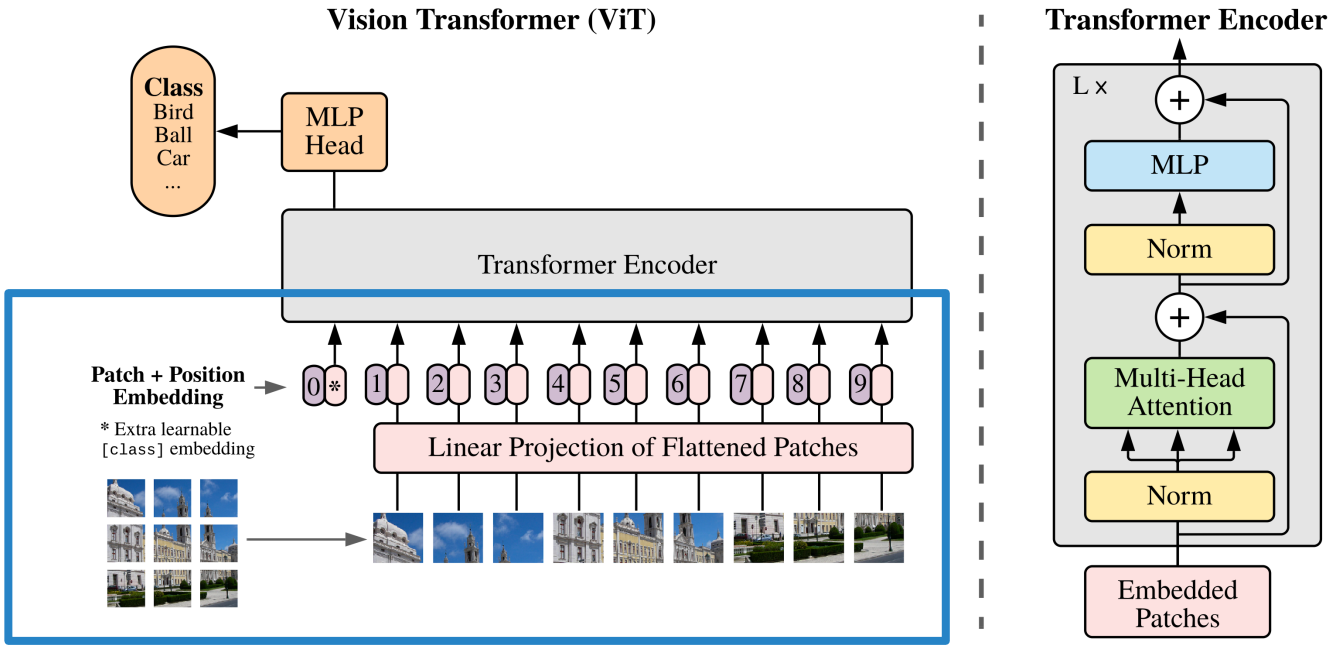


Goals

- In this assignment, you will learn how to:
  - Design an AI accelerator with quantization support
  - Implement multi-bit Clock Domain Crossing (CDC) techniques
  - Interface with external memory via the AXI bus

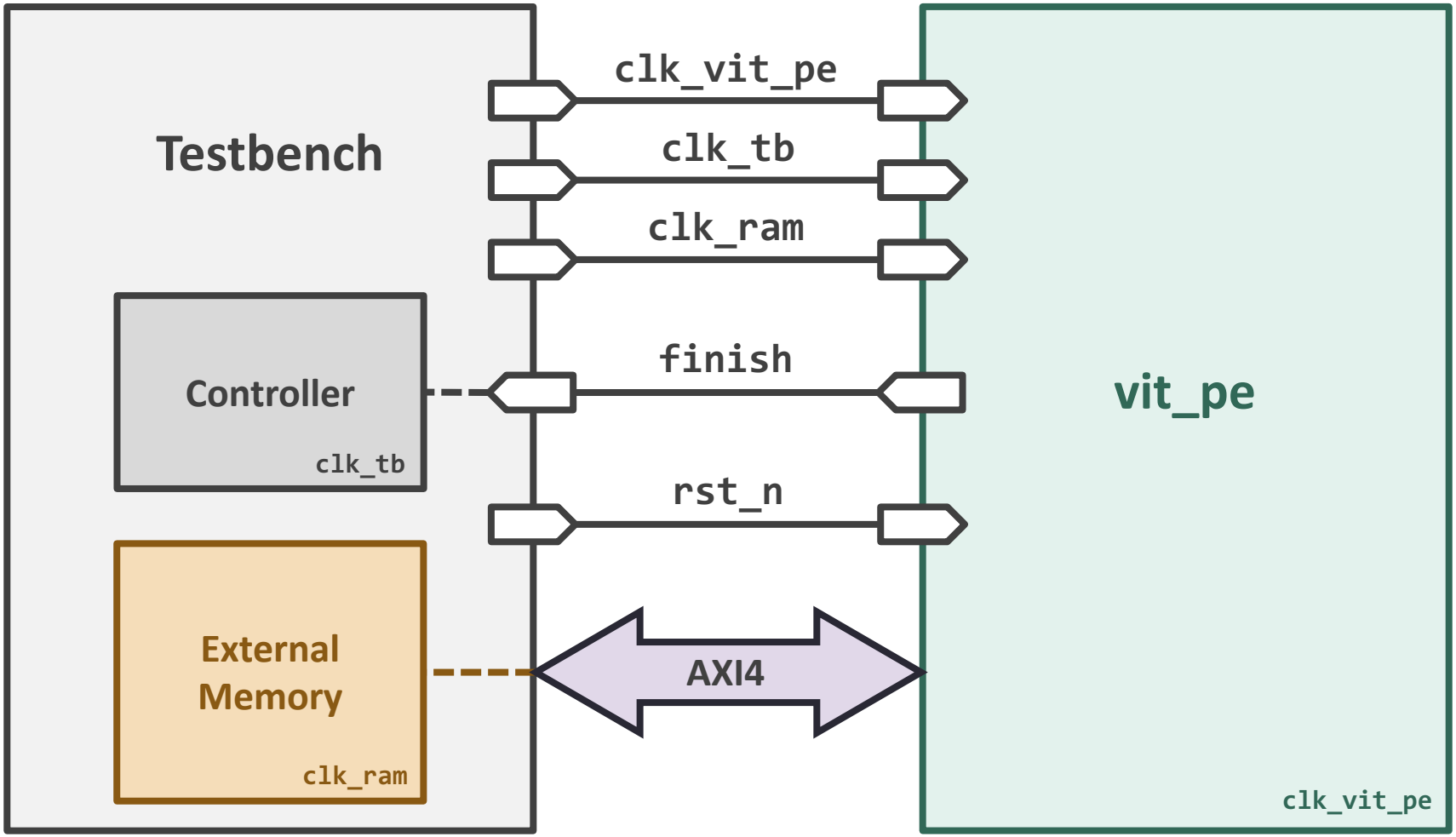
Introduction

- In this project, you will implement a Patch Embedding accelerator for a Vision Transformer (ViT) model



You will implement this part

Block Diagram



Input/Output

Signal Name	I/O	Width	Simple Description
`clk_tb`	I	1	• Primary clock for the testbench **(positive edge-triggered)** • Inputs are synchronized with a half-cycle delay
`clk_vit_pe`	I	1	• Design clock for the accelerator **(positive edge-triggered)**
`clk_ram`	I	1	• Clock for AXI channels and memory access **(positive edge-triggered)** • All AXI inputs/outputs must be synchronized with this clock's rising edge
`rst_n`	I	1	• **Active-low synchronous reset**
`finish`	O	1	• Computation completion flag • Must remain HIGH for at least three `clk_tb` cycles to signal termination

Input/Output

Signal Name	I/O	Width	Simple Description
awaddr araddr	O	15	• Address Channel (AW/AR) • Specifies the start address of the first transfer in a burst
awlen arlen	O	8	• Burst Length • Number of data transfers within a burst (*N* = *len* + 1)
awsize arsize	O	3	• Burst Size • Specifies the size (bytes) of each individual transfer
awburst arburst	O	2	• Burst Type • Determines how the address increments for each transfer
awvalid arvalid	O	1	• Address Valid • Indicates that the address and control information are valid
awready arready	I	1	• Address Ready • Indicates that the slave is ready to accept the address

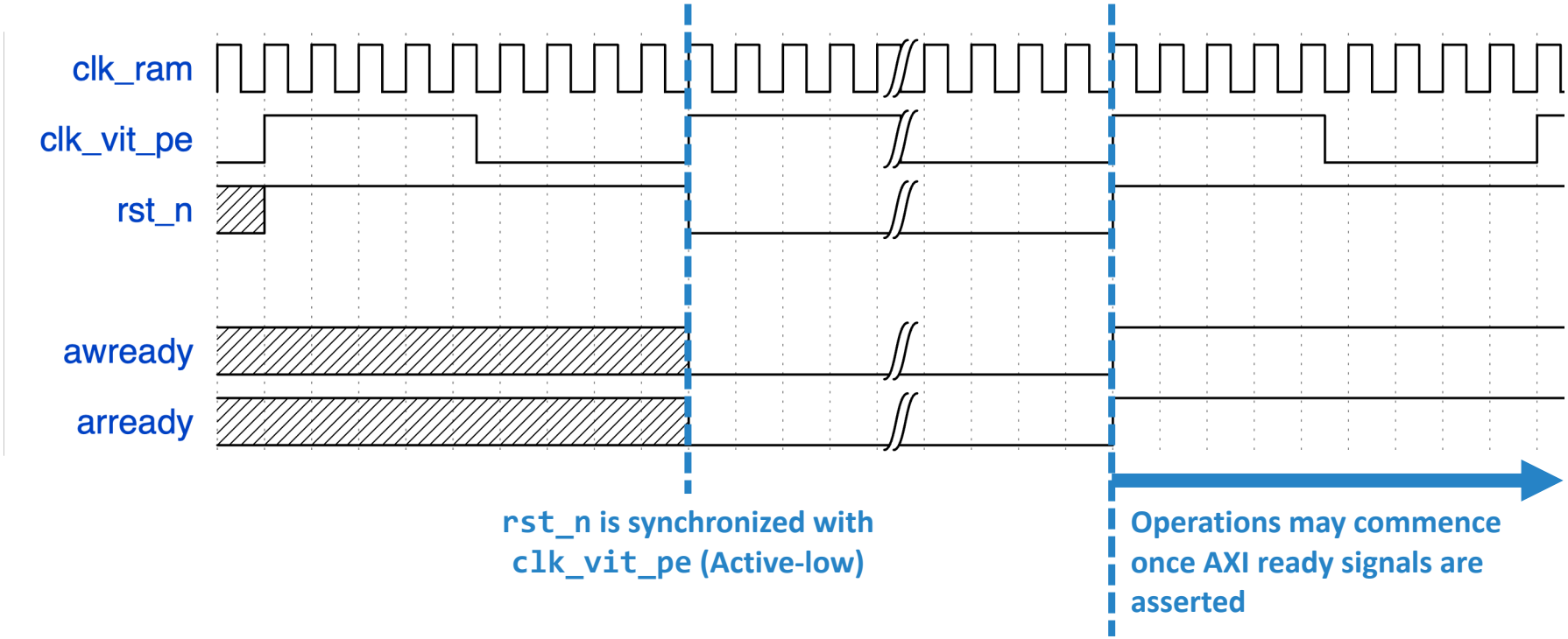
Input/Output

Signal Name	I/O	Width	Simple Description
wdata rdata	O I	8	• Data Channel (W/R) • Write/Read data payload
wlast rlast	O I	1	• Last Transfer Indicator • Indicates the last transfer in a burst
wvalid rvalid	O I	1	• Data Valid • Indicates that the data/strobe is valid
wready rready	I O	1	• Data Ready • Indicates that the master/slave can accept data
wstrb	O	1	• Write Strobes • Indicates which byte lanes contain valid data
bresp rresp	I	2	• Response Status • Status of the write/read transfer (e.g., 2'b00 for OKAY)
bvalid bready	I O	1	• Write Response Channel (B) • Handshake signals for the write response

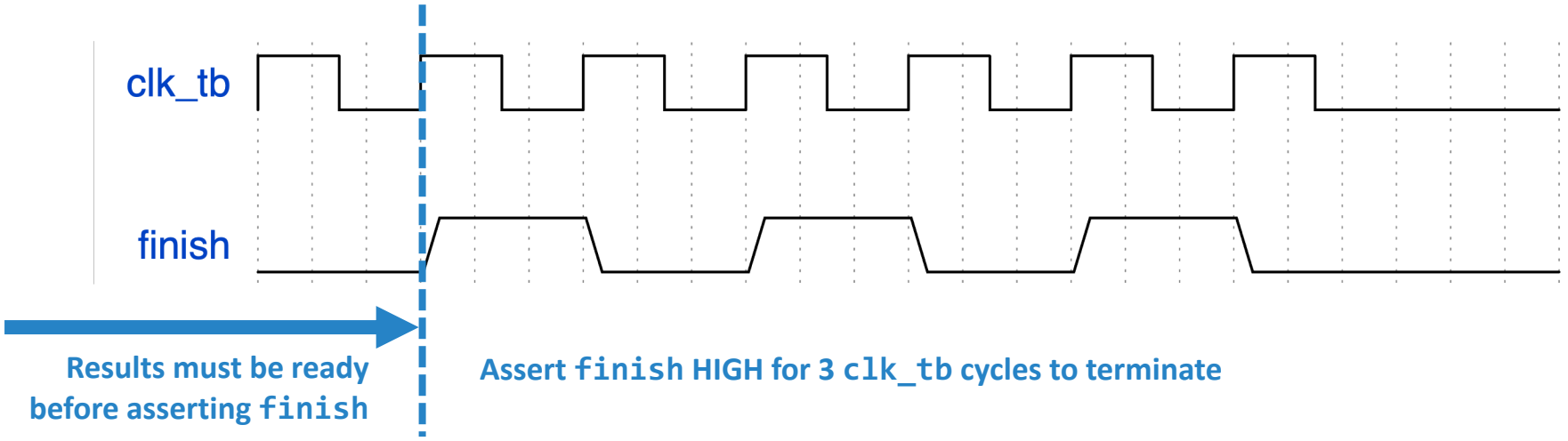
Specification (1/3)

- System Reset & Initialization:
  - `rst_n` is synchronized with the positive edge of of `clk_vit_pe`
  - Test patterns (Image and Weights) are pre-loaded into the external memory after the reset signal is de-asserted
- Clock Domain & Synchronization:
  - All AXI input and output signals must be strictly synchronized with the positive edge of `clk_ram`
  - **CDC techniques should be properly implemented**
- Operation Trigger:
  - No explicit start signal is provided in this project; the accelerator begins operations autonomously once the memory is ready post-reset
  - The `finish` signal must be synchronized with `clk_tb` and remain HIGH for three cycles to indicate the end of the entire computation task
- DesignWare and ADFP SRAM are allowed

Specification (2/3)

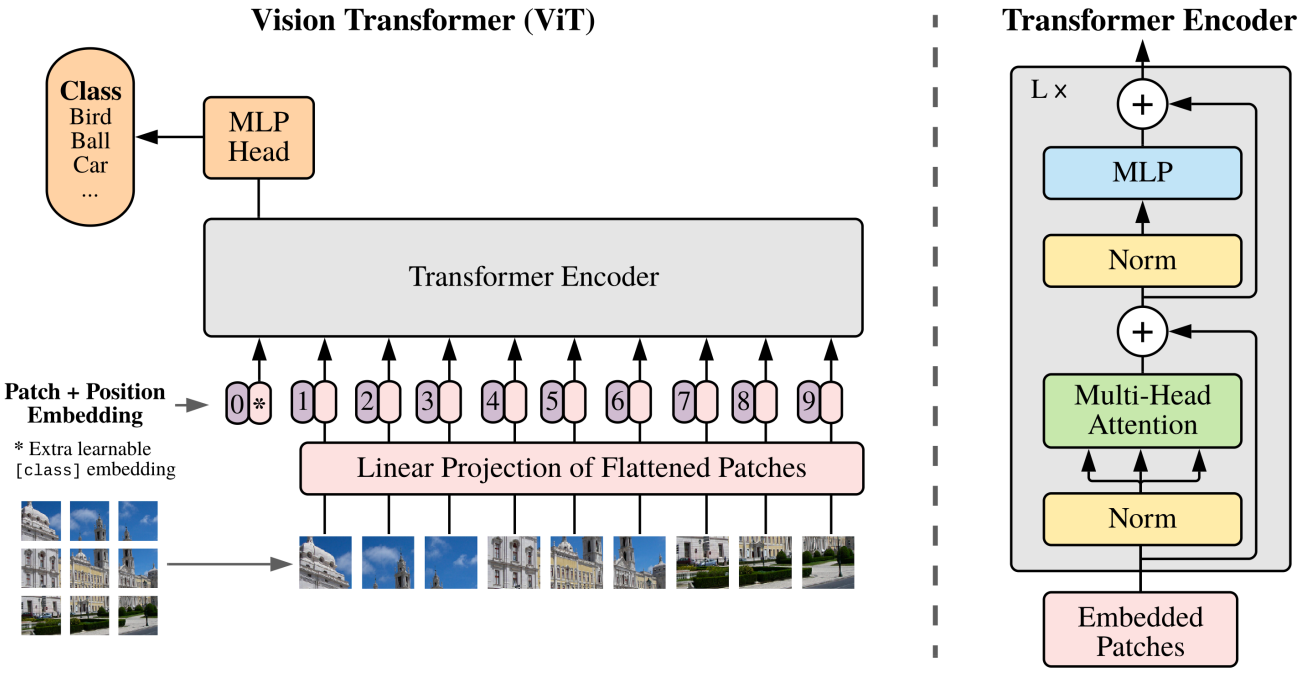


Specification (3/3)



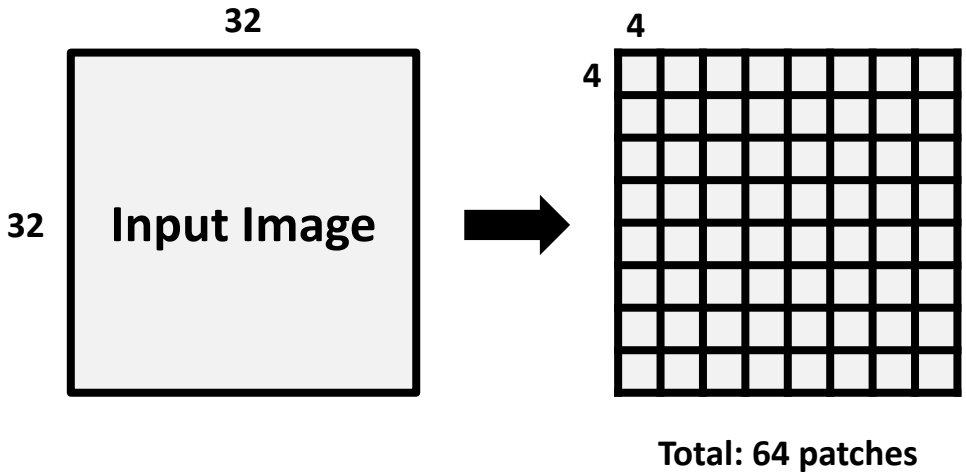
Vision Transformer (ViT) (1/2)

- A revolutionary architecture that treats images like sentences
- Instead of processing pixels with CNNs, ViT splits an image into a sequence of fixed-size "patches" and feeds them into a Transformer encoder



Vision Transformer (ViT) (2/2)

- Original ViT typically uses 16 × 16 patches
- Our hardware design will be optimized for 32 × 32 RGB images (Unsigned INT8), compatible with CIFAR-10 dimensions
- We split the 32 × 32 image into **4 × 4 patches**, resulting in a sequence length of **64 patches**

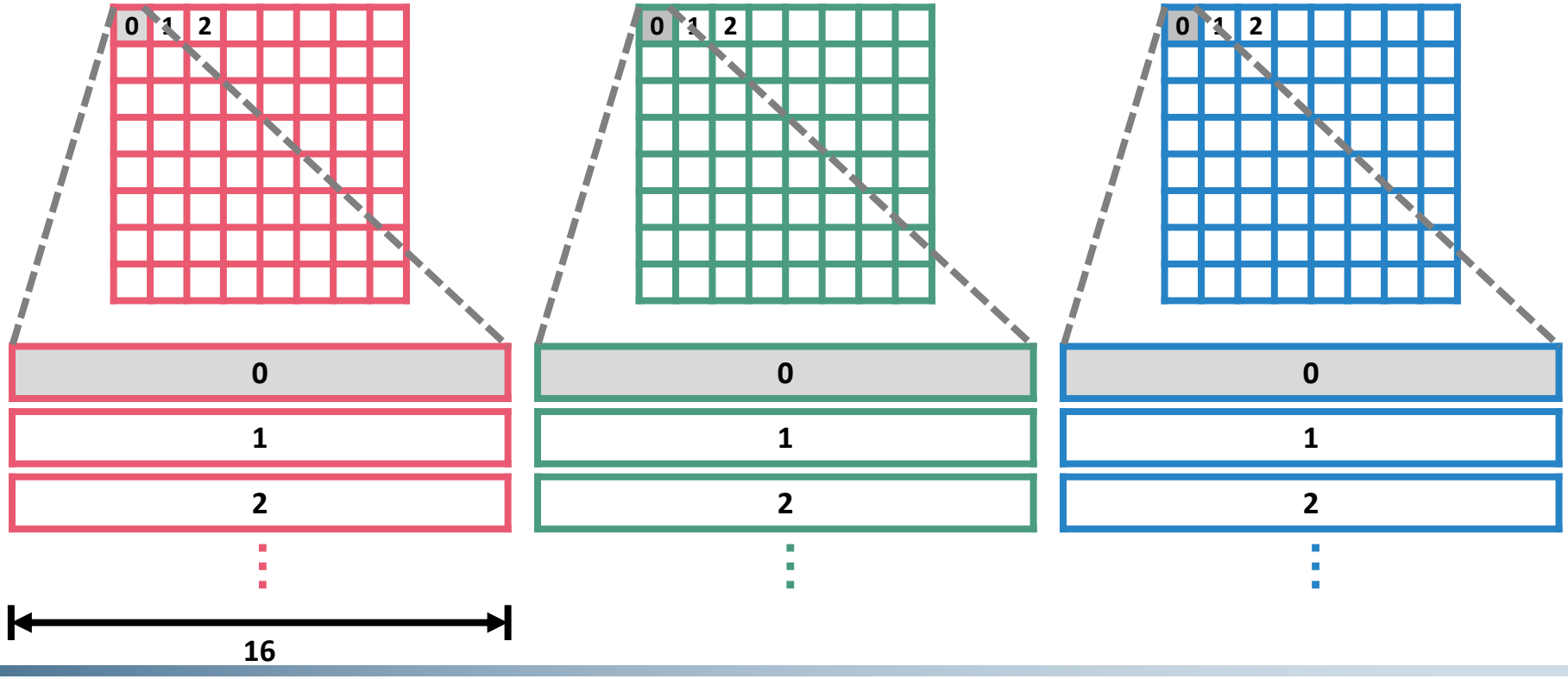


Workflow

1. **Image Fetch:** Load image data from memory, split it into 4 × 4 patches, and flatten each patch
2. **Linear Projection**: Perform matrix multiplication (Patch → Embedding)
3. **Quantization**: Quantize the high-precision accumulator back to INT8
4. **CLS Token**: Append the learnable Class (CLS) Token
5. **Position Embedding**: Add Position Embeddings to retain spatial awareness and clip the final result

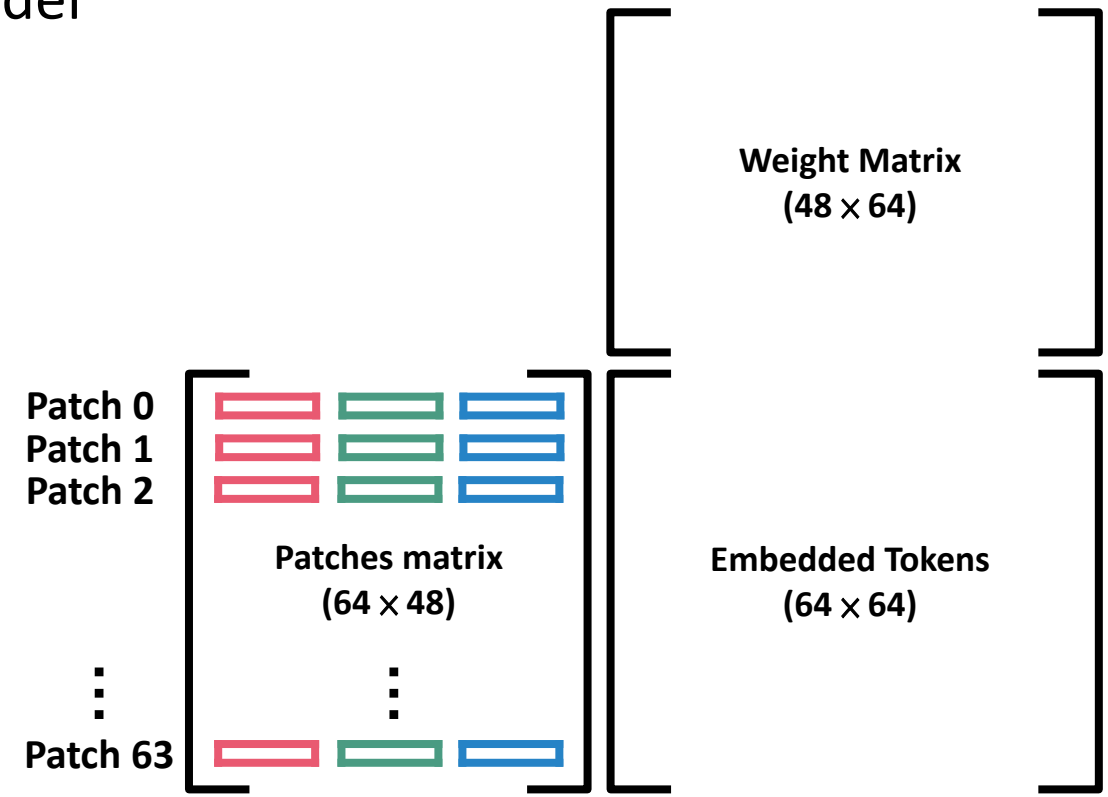
Image Fetch

- Load image, split into 4 × 4 patches, and flatten into 1D vectors
- **Format of image data: Unsigned INT8**, stored in raster scan order, separated by channel (R → G → B)



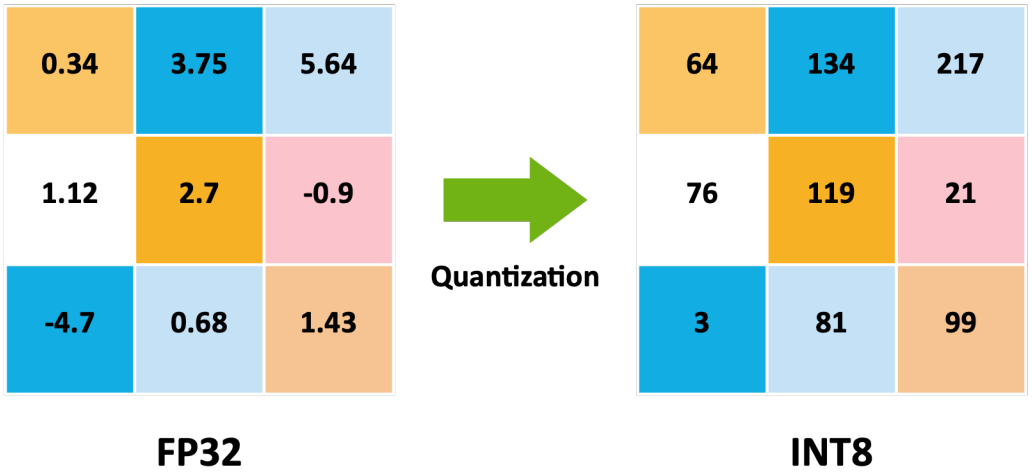
Linear Projection

- Flattened patches matrix (64 × 48) is multiplied by the weight matrix (48 × 64) to map patches into the embedding dimension
- **Format of weight matrix data: Signed INT8**, stored in raster scan order

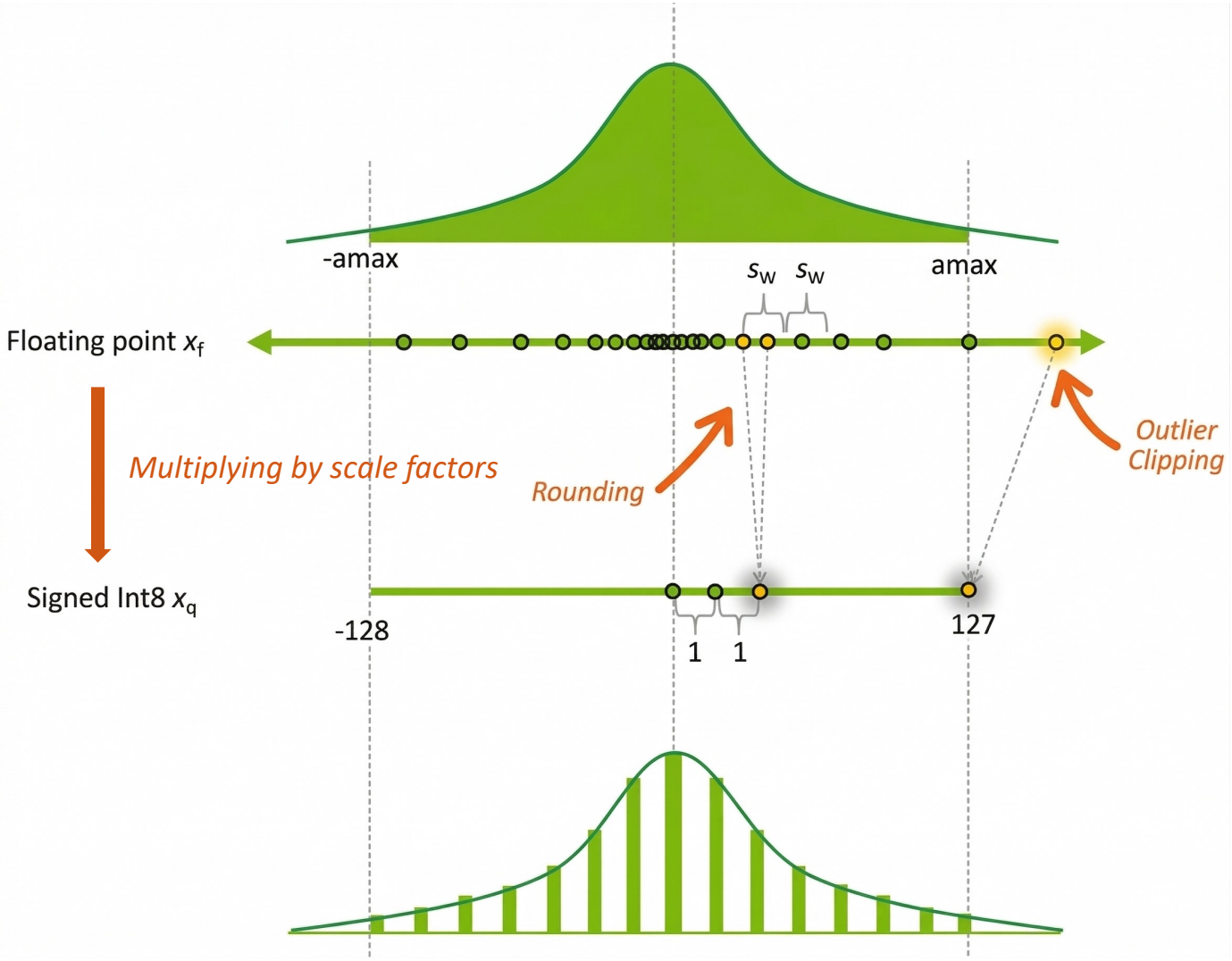


Background of Quantization (1/2)

- Lower bit-width significantly reduces hardware complexity and power consumption
  - 8-bit integer (INT8) operations are vastly more energy-efficient than 32-bit floating-point (FP32)
- A well-trained model suffers minimal performance degradation when FP32 operations are replaced by INT8



Background of Quantization (2/2)



Quantization

- **Format of scale factors:**
  - In this project, scale factors are pre-computed offline as **Unsigned 32-bit fixed-point** to scale the linear projection results down to the INT8 range
  - All 32 bits are fraction bits (a binary scaling of 2^{-32})
  - A 32-bit scale factor is split into four 8-bit bytes, stored sequentially from MSB to LSB in memory

$(0.75)_{10} = (0.5+0.25)_{10} = (0.11)_2$, stored as "11000000" "00000000" "00000000" "00000000"

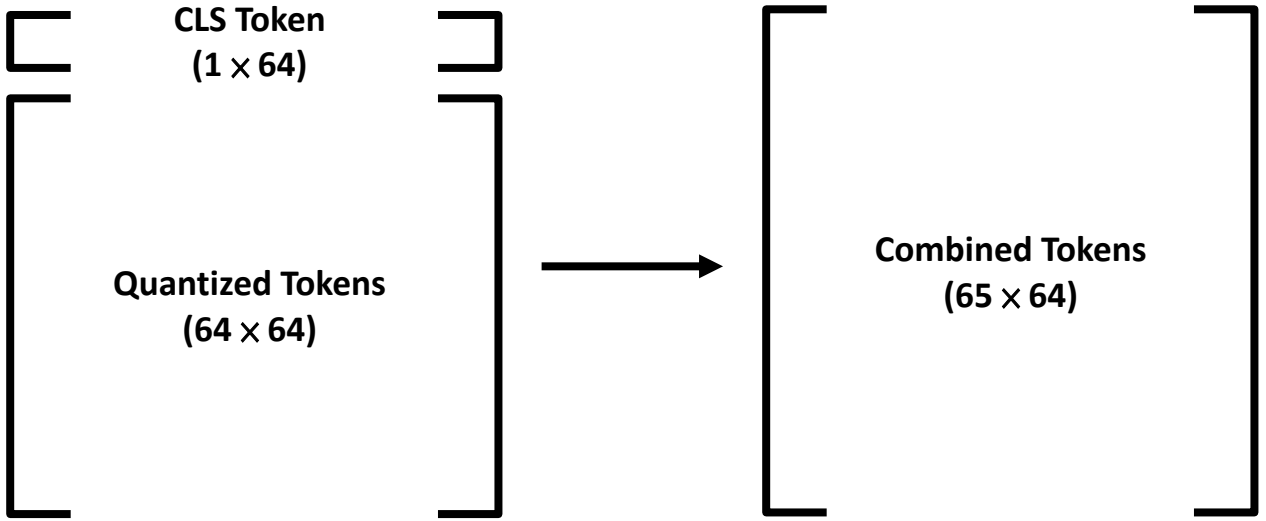
mem[x] mem[x+1] mem[x+2] mem[x+3]

- **Quantization Steps:**
  1. **Scale & Shift:** Multiply each column of the matrix (representing a single embedding dimension across all 64 tokens) by its dedicated scale factor
  2. **Round:** Round the result to **the nearest integer**
  3. **Clipping is not needed at this stage; we will delay it until after adding Position Embeddings**

$$j\text{-th_column}_{\text{INT8}} = \text{round}\left(j\text{-th_column} \cdot \text{scale_factor}[j]\right), \quad j = 0, 1, 2, \cdots, 63$$

CLS Token Fetch

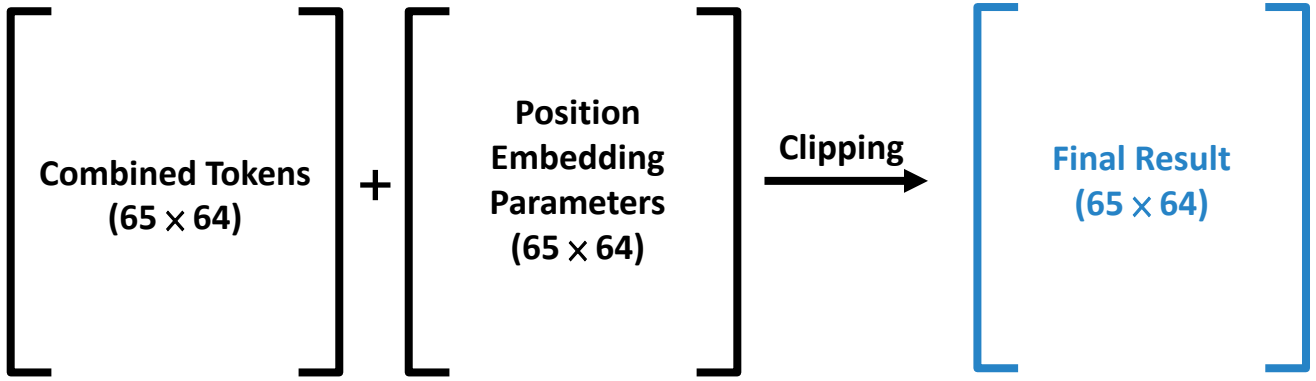
- CLS Token (1 × 64) is a special token prepended to the sequence
- Its final output represents the global image classification
- **Format of CLS Tokens: Signed INT8**



Position Embedding

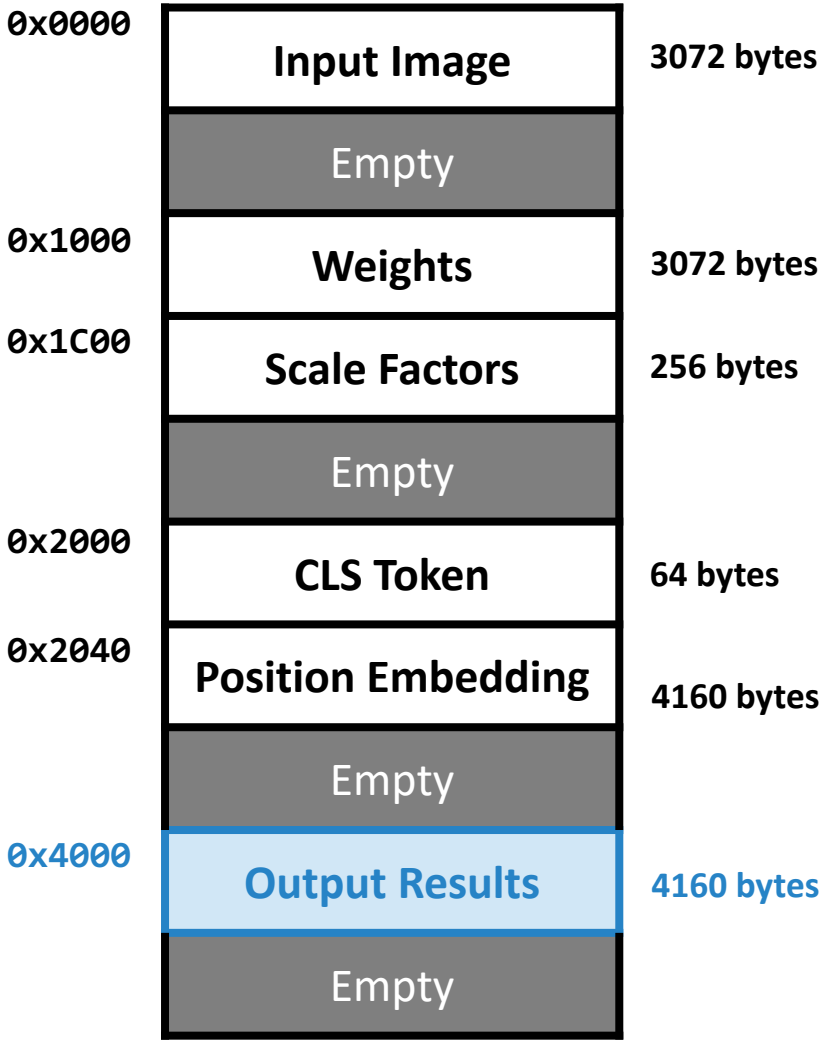
- Since Transformers lack inherent sequence order, positional data are added to tokens to retain spatial awareness.
- **Format of position embedding: Signed INT8**, stored in raster scan order
- Summed results should be saturated (clipped) to Signed INT8
  - Let the matrices to add be **A**, **B**, and the clipping result be **C**.

$$\mathbf{C}[i][j] = \begin{cases} 127, & \text{if } \mathbf{A}[i][j] + \mathbf{B}[i][j] > 127, \\ -128, & \text{if } \mathbf{A}[i][j] + \mathbf{B}[i][j] < -128, \\ \mathbf{A}[i][j] + \mathbf{B}[i][j], & \text{otherwise} \end{cases} \quad i = 0, 1, \cdots 64,\ j = 0, 1, \cdots 63$$

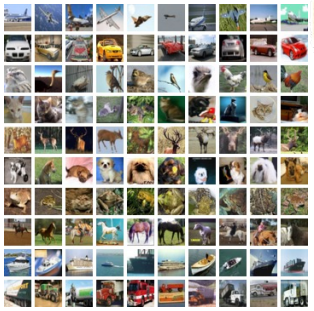


External Memory

- The memory has a 32KB total capacity with an 8-bit word size
- All input data and model parameters must be fetched from their designated base addresses
- **Final embedding results must be written starting at base address `0x4000`**
- Memory regions that are not verified by the testbench can be used to store intermediate computational results



Patterns



Source: https://datasets.cms.waikato.ac.nz/ufdl/cifar10/

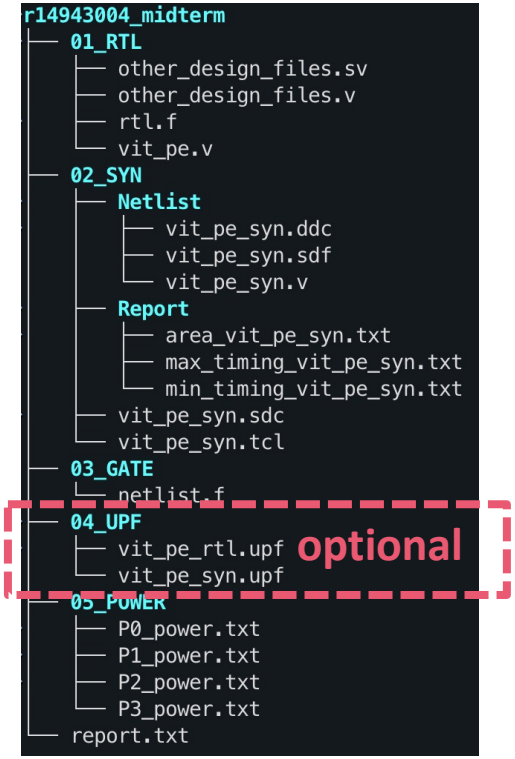
- Public Test Patterns (P0, P1, P2, P3):
  - Sourced from the CIFAR-10 dataset
  - Taking pattern P0 as an example:
    - `P0_horse.png`: The original image for visual reference
    - `P0_picture.dat`: Input image in hex **(Mapped to `0x0000-0x0BFF`)**
    - `P0_weights.dat`: Parameters in hex **(Mapped to `0x1000-0x307F`)**
    - `P0_golden.dat`: Expected output results **(Checked at `0x4000-0x503F`)**



- Private Test Patterns (Hidden):
  - May use images outside the CIFAR-10 dataset
  - Will strictly adhere to the exact same dimensions and formats as the public patterns

Submission

- Create a folder named exactly as **`<studentID>_midterm`** (e.g., `r14943004_midterm`)
  - Both standard Verilog (`.v`) and SystemVerilog (e.g. `vit_pe.sv`) are fully allowed in the `01_RTL` directory
  - `04_UPF` is optional



Submission - Workstation

- Pack the folder **<studentID>_midterm** into a **.tar.gz file** named **acvsd-xx-midterm.tar.gz**
  - **tar -zcvf acvsd-xx-midterm.tar.gz <path to <studentID>_midterm>**
  - Use **lowercase** for all the letters (e.g. `acvsd-00-midterm.tar.gz`)
  - Place the `.tar.gz` file at the home (~) of your ADFP account

- TA will only check the latest version after the homework deadline
- **Files uploaded to the ADFP workstation must be kept as a copy in your local folder or computer**

Submission - NTU COOL

- For RTL design files (`01_RTL`), `.sdc` and `.tcl` files (02_SYN) and optional UPF files (`04_UPF`)
- Pack the folder **`<studentID>_midterm`** into a **`.tar.gz` file** named **`acvsd-xx-midterm-v`*`k`*`.tar.gz`** (*k* is the number of version, *k* =1,2,…)
  - **`tar -zcvf acvsd-xx-midterm-v`*`k`*`.tar.gz <path to <studentID>_midterm>`**
  - Use **lowercase** for all the letters. (e.g. `acvsd-00-midterm-v1.tar.gz`)
- Submit to NTU COOL

```
r14943004_midterm
├── 01_RTL
│   ├── other_design_files.sv
│   ├── other_design_files.v
│   ├── rtl.f
│   └── vit_pe.v
├── 02_SYN
│   ├── vit_pe_syn.sdc
│   └── vit_pe_syn.tcl
├── 04_UPF
│   └── vit_pe_rtl.upf
└── report.txt
```

Grading Policy (1/3)

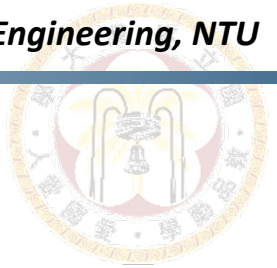
- **Simulation: 80%**
  - Zero runtime timing violations are allowed after reset de-assertion.
  - **Total simulation time must not exceed 2.5×10^6 ns (2.5 ms)**

	Public must pass all patterns to receive any score	**Private** graded proportionally based on the pass rate
RTL	15%	15%
Gate-level	25%	25%

- **Performance: 20%**

$$\text{score} = \left(\sum_{i=0}^{3} \text{power}_i \times \text{time}_i \right) \times \text{area}^2$$

  - Unit: power (mW), time (ns), area (μm^2)
  - There is no hard baseline for performance scores in this assignment
  - You are encouraged to challenge yourself to optimize hardware area



Grading Policy (2/3)

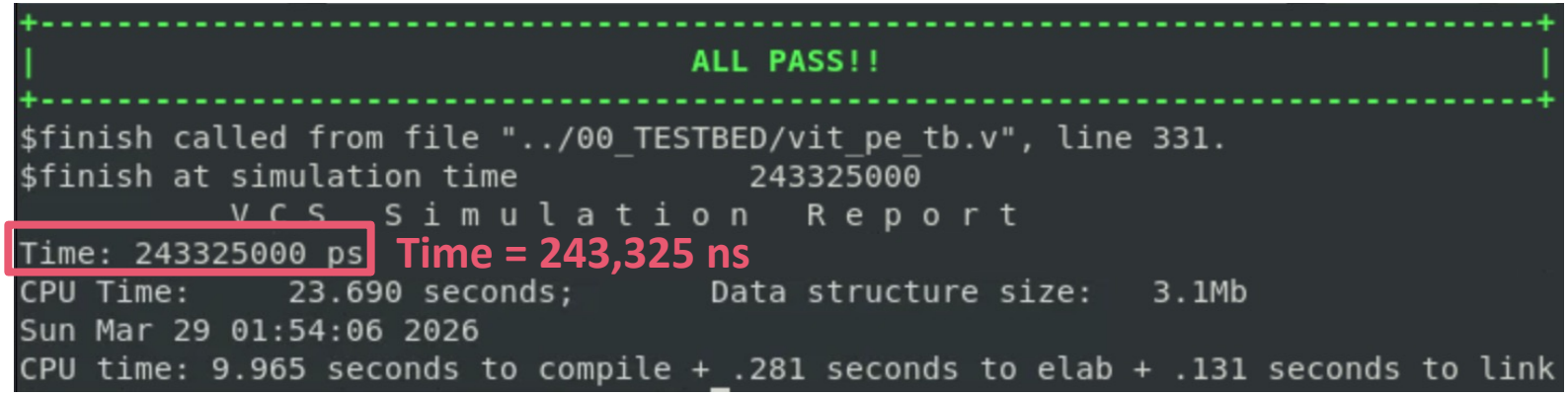
- Grading command for RTL simulation
  - **`vcs -full64 -R -f rtl.f -sverilog -v2005 +v2k +define+PAT_NUM=i +notimingcheck`**
  - *i* is the number of pattern
- Grading command for Gate-Level simulation
  - **`vcs -full64 -R -f netlist.f +v2k -sverilog -v2005 +define+SDF+PAT_NUM=i +maxdelays -negdelay +neg_tchk`**
  - *i* is the number of pattern
- You can `+define+FSDB` and add `–debug_access+all` to dump `.fsdb` file

Grading Policy (3/3)

- -5 points for any incorrect file naming or directory structure
- Specification Violations
  - Negative setup/hold slack
    - 0 point for gate-level simulations and performance sections
  - Other Violations (with passing all simulations)
    - Performance score will be multiplied by 0.7
- TA will run simulation with UPF if submitted
- **No late submission**
  - **0 point** for this assignment
- **No plagiarism**
  - Plagiarism in any form, including copying from online sources, is strictly prohibited

Time

- Extract the total simulation time directly from the VCS simulation report (e.g. 243,325 ns below)
- Note: The figure provided is for reference only



Area

- Extract the **Total cell area** from the area report
- Note: The figure provided is for reference only

```
Number of ports:                          114
Number of nets:                          1975
Number of cells:                         1827
Number of combinational cells:           1715
Number of sequential cells:               112
Number of macros/black boxes:               0
Number of buf/inv:                        307
Number of references:                     127

Combinational area:              23381.684796
Buf/Inv area:                     1573.489784
Noncombinational area:            4498.109936
Macro/Black Box area:                0.000000
Net Interconnect area:      undefined  (No wire load specified)
```

Area = 27879.794732 μm²

```
Total cell area:                 27879.794732
Total area:                 undefined
```

Power

- Use PrimeTime to evaluate power based on the `.vcd` file generated by `03_fsdb2vcd.tcsh`
- Note: The figure provided is for reference only

```
  Attributes
  ----------
      i  -  Including register clock pin internal power
      u  -  User defined power group

                        Internal  Switching  Leakage    Total
Power Group             Power     Power      Power      Power   (     %)  Attrs
--------------------------------------------------------------------------------
clock_network           2.160e-04    0.0000    0.0000 2.160e-04 (39.30%)  i
register                4.534e-05 2.469e-05 3.130e-06 7.316e-05 (13.31%)
combinational           1.586e-04 9.146e-05 1.037e-05 2.605e-04 (47.39%)
sequential                 0.0000    0.0000    0.0000    0.0000 ( 0.00%)
memory                     0.0000    0.0000    0.0000    0.0000 ( 0.00%)
io_pad                     0.0000    0.0000    0.0000    0.0000 ( 0.00%)
black_box                  0.0000    0.0000    0.0000    0.0000 ( 0.00%)

  Net Switching Power  = 1.161e-04   (21.13%)
  Cell Internal Power  = 4.200e-04   (76.41%)
  Cell Leakage Power   = 1.350e-05   ( 2.46%)
                         ---------
Total Power            = 5.496e-04  (100.00%)
```

Power = 0.0005496 mW

Report

- Please complete and submit the `report.txt` file.
- Scores are based solely on TA's runs, not the self-reported numbers in this file.

```
StudentID: r14943004

Clock period: 5.0    (ns  )
Area: 98765.432100   (um^2)

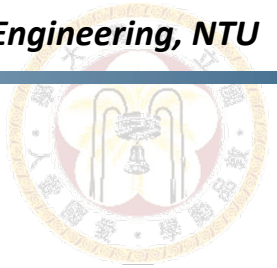
Is UPF used? (T/F) F

--------------------------
P0 Time:  456789.10  (ns  )
P0 Power: 0.8765     (mW  )
--------------------------
P1 Time:  345678.91  (ns  )
P1 Power: 0.7654     (mW  )
--------------------------
P2 Time:  234567.89  (ns  )
P2 Power: 0.6543     (mW  )
--------------------------
P3 Time:  123456.78  (ns  )
P3 Power: 0.5432     (mW  )
--------------------------
```

Discussion

- **NTU Cool Discussion Forum**
  - For any questions not related to assignment answers or privacy concerns, please use the NTU Cool discussion forum.
  - **TAs will prioritize answering questions on the NTU Cool discussion forum**

- **Email:** r14943004@ntu.edu.tw
  - Title should start with **[ACVSD 2026 Spring Midterm]**
  - Email with wrong title will be moved to trash automatically
  - You need to identify yourself when writing



References

[1] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021. [Online]. Available: https://arxiv.org/abs/2010.11929